```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define N 100

// Global stack and top variable
char stack[N];
int top = -1;

// Missing stack functions added for completeness
void push(char item)
{
    if(top >= N - 1)
    {
        printf("\nStack Overflow");
    }
    else
    {
        top++;
        stack[top] = item;
    }
}

char pop()
{
    if(top == -1)
    {
        return '\0'; // Return null character if underflow
    }
    else
    {
        char item = stack[top];
        top--;
        return item;
    }
}

void peek()
{
    if(top == -1)
    {
        printf("\nStack is empty");
    }
    else
    {
        printf("\nTop element is %c", stack[top]);
    }
}
```

```
int icp(char ch)
{
    if(ch=='+' || ch=='-') { return 1; }
    else if(ch=='*' || ch=='/') { return 4; }
    else if(ch=='^') { return 5; }
    else if(ch=='(') { return 6; }
    else { return 0; }
}

int isp(char ch)
{
    if(ch=='+' || ch=='-') { return 1; }
    else if(ch=='*' || ch=='/') { return 2; }
    else if(ch=='^') { return 3; }
    else { return 0; }
}

void in_post(char inexp[])
{
    int k=0, i=0;
    char tkn = inexp[i];
    char postexp[N];

    // Reset stack for conversion
    top = -1;

    while(tkn != '\0')
    {
        if(isalnum(tkn))
        {
            postexp[k]=inexp[i];
            k++;
        }
        else
        {
            if(tkn=='(')
            {
                push('(');
            }
            else if(tkn==')')
            {
                while((tkn=pop())!='(')
                {
                    postexp[k]=tkn;
                    k++;
                }
            }
```

```
            else
            {
                while(top!=-1 && isp(stack[top])>=icp(tkn))
                {
                    postexp[k]=pop();
                    k++;
                }
                push(tkn);
            }
        }
        i++;
        tkn=inexp[i];
    }
    while(top!=-1)
    {
        postexp[k]=pop();
        k++;
    }
    postexp[k] = '\0'; // Null terminate the string
    printf("\nPostfix Expression: %s\n", postexp);
}

int main()
{
    int choice;
    char item;
    char expression[N];

    do{
        printf("\n----MENU----\n");
        printf("1.Push\n2.Pop\n3.Peek\n4.Display\n5.ICP (Incoming
Precedence)\n6.ISP (In-Stack Precedence)\n7.Infix to Postfix
Conversion\n8.Exit\n");
        printf("Choose the operation you want to perform = ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter character to push: ");
                scanf(" %c", &item);
                push(item);
                break;
            case 2:
                item = pop();
                if(item != '\0') printf("Popped: %c\n", item);
                break;
            case 3:
                peek();
                break;
```